

TechCheck Pro — Documentación del Proyecto

Módulo de Inferencia de Adopción FLOSS (TOE)

Basado en la guía GUIOS de la tesis *Factores para la adopción de soluciones basadas en software libre* (Capítulos V y VI).

Este documento explica cómo funciona la aplicación de punta a punta: autenticación, origen de los datos, cálculos del Capítulo V, motor FODA, dictamen, despliegue en producción (Vercel + Neon), relación con el prototipo GUIOS PRO (guiosad.html) y el estado actual de la implementación.

Última actualización: 2026-07-12

Tabla de contenidos

1. ¿Qué es TechCheck Pro?
2. Relación con GUIOS PRO (guiosad.html)
3. Arquitectura general
4. Autenticación y login (RF-01)
5. Flujo de uso del evaluador
6. De dónde salen los datos: CSV, tesis y base de datos
7. Los 5 indicadores de importancia (Capítulo V)
8. Pantalla de evaluación: decisor, IR y subfactores
9. Evaluación de subfactores (escala Likert)
10. Cálculos del motor GUIOSAD paso a paso
11. Matriz FODA (Ecuación 5.3 de la tesis)
12. Dictamen final: Clase A, B y C
13. Paridad backend ↔ frontend ↔ GUIOS PRO
14. Verificación Cap. V vs implementación actual
15. API REST disponible
16. Modelo de datos (PostgreSQL)
17. Comandos útiles para desarrollo
18. Solución de problemas frecuentes
19. Despliegue en producción (Vercel + Neon)

1. ¿Qué es TechCheck Pro?

TechCheck Pro es la evolución web del prototipo GUIOS PRO descrito en la tesis. Automatiza la guía GUIOS (Guía para la adopción de FLOSS) mediante:

- Un catálogo TOE fijo: **3 dimensiones, 18 factores, 61 subfactores**.
- Un wizard donde el evaluador ajusta la importancia del decisor por factor y califica subfactores.
- Un motor de inferencia (**MotorGUIOSAD**) que calcula IR, clasifica factores en FODA y emite un dictamen de adopción.
- Persistencia en **PostgreSQL** (a diferencia del sistema anterior que usaba CSV + HTML estático sin historial).

Sistema anterior (GUIOS PRO / guiosad.html)	TechCheck Pro (actual)
---	------------------------

Cargar CSV manualmente cada sesión	CSV → seed_toe → PostgreSQL (una vez)
Motor en ActionScript/Flex embebido en HTML	MotorGUIOSAD en Django + utilidades JS espejo
Sin login ni historial	JWT + portafolio de evaluaciones
Sliders y botones en una sola página	Wizard en 3 pestañas con autosave
Misma lógica de IR, FODA y dictamen	Réplica verificada con 14 tests

Principio de diseño: los cálculos deben ser idénticos al GUIOS PRO anterior; solo cambia la interfaz y la persistencia.

2. Relación con GUIOS PRO (guiosad.html)

El archivo guiosad.html en la raíz del repositorio es la referencia funcional del motor. TechCheck Pro no reinventa las fórmulas: las centraliza en módulos dedicados que replican ese comportamiento.

Concepto	GUIOS PRO (guiosad.html)	TechCheck Pro
Paso 1–2	Slider Evaluación del decisor (ID) 1–4	Botones Irrelevante / Opcional / Importante / Fundamental
IS	Fija, viene del catálogo	Factor.importancia_sugerida (desde factors.csv)
IR	<code>floor((idx(IS) + idx(ID)) / 2)</code>	<code>guiosad_calculos.py</code> / <code>guiosadCalculos.js</code>
Factores irrelevantes	No muestra subfactores	Oculto subfactores si IR = Irrelevante
Likert subfactores	Escala 1–4 (PS)	Igual
FODA	$PM \geq 3$ positivo; $PM < 3$ negativo	<code>clasificar_foda()</code>
Dictamen	A=Adoptar, B=Condicionado, C=Rechazar	<code>resolver_dictamen()</code> — misma regla

Archivos de referencia de cálculo:

Archivo	Rol
<code>guiosad.html</code>	Prototipo original (referencia visual y lógica)
<code>backend/core/utils/guiosad_calculos.py</code>	Fórmulas puras (IR, FODA, dictamen)
<code>backend/core/utils/guiosad_engine.py</code>	Orquestación sobre PostgreSQL
<code>frontend/src/utils/guiosadCalculos.js</code>	Misma lógica IR en la UI (cálculo en vivo)

3. Arquitectura general

Desarrollo local (Docker Compose)

FRONTEND
(React + Vite + Tailwind) – http://localhost:3000 ■ Login → Panel de Control → Matriz de Evaluación
→ Dictamen/PDF ■
■ HTTP + JWT Bearer

BACKEND
(Django REST Framework) – http://localhost:8000/api ■ Auth JWT ■ CRUD evaluaciones ■ Autosave ■
MotorGUIOSAD ■
ORM ■ PostgreSQL
15 (contenedor Docker) ■ Catálogo TOE ■ Respuestas Likert ■ Detalles por factor ■ Dictamen■

Producción (Vercel + Neon)

```
graph TD; subgraph FRONTEND; F["(React/Vite) - https://techcheck-pro.vercel.app"]; end; subgraph BACKEND; B["(Django serverless) - misma URL /api/"]; end; subgraph DB["PostgreSQL (Neon - serverless)"]; end; F -- "/api/*" --> B; B -- "SSL" --> DB; DB -- "SSL" --> B;
```

Stack: Django REST Framework, PostgreSQL, React (Vite 8), JWT (simplejwt), Docker Compose (desarrollo), Vercel Services + Neon (producción).

Contenedores locales:

Servicio	Puerto	Comando
frontend	3000	<code>npm install && npm run dev -- --host 0.0.0.0 --port 3000</code>
backend	8000	<code>python manage.py runserver 0.0.0.0:8000</code>
db	5432	PostgreSQL 15

URLs de producción:

Recurso	URL
Aplicación	https://techcheck-pro.vercel.app
Login	https://techcheck-pro.vercel.app/login
API	https://techcheck-pro.vercel.app/api/
Dashboard Vercel	https://vercel.com/techgamingec-5949s-projects/techcheck-pro

4. Autenticación y login (RF-01)

¿Cómo funciona?

1. El usuario ingresa usuario y contraseña en `/login`.
2. El frontend llama a `POST /api/token/` con `{ username, password }`.
3. Django devuelve un par de tokens JWT:
 - `access` — token de acceso (8 horas).
 - `refresh` — token para renovar el `access` (7 días).
4. Se guardan en `localStorage`: `access`, `refresh`, `username`.
5. Cada petición al backend lleva el header: `Authorization: Bearer <access>`.
6. Si el `access` expira, el interceptor de Axios en `frontend/src/services/api.js` renueva automáticamente con `/api/token/refresh/`.
7. Al cerrar sesión se limpian tokens, `username` y `activeEvalId`.

Roles

Rol	Descripción
EVALUADOR	Crea evaluaciones, califica subfactores, calcula dictamen
ADMIN	Gestión de usuarios y catálogo TOE (RF-02)

Usuarios de referencia

Desarrollo local:

```
docker compose run --rm backend python manage.py create_evaluador # Usuario: evaluador / evaluador123
```

Producción (Vercel):

```
Usuario: admin Contraseña: Admin2026!
```

(Creado automáticamente por `build.py` durante el despliegue)

Archivos clave

Archivo	Responsabilidad
<code>frontend/src/pages/Login.jsx</code>	Pantalla de login
<code>frontend/src/context/AuthContext.jsx</code>	Estado global de sesión
<code>frontend/src/App.jsx</code>	<code>ProtectedRoute</code> — redirige a login si no hay sesión
<code>backend/core/urls.py</code>	Rutas <code>/api/token/</code> y <code>/api/token/refresh/</code>

5. Flujo de uso del evaluador

La aplicación tiene 3 pestañas principales (wizard):

Pestaña 1 — Panel de Control & Alcance (RF-03)

- Registrar un software objetivo (nombre, versión, proveedor).
- Ver historial de evaluaciones del usuario (GET /api/evaluaciones/misfichas/).
- Al crear/seleccionar un proyecto se llama POST /api/evaluaciones/iniciar/.
- Se guarda activeEvalId en localStorage para continuar entre pestañas.
- Al iniciar una evaluación, el backend crea automáticamente:
- 18 registros en DETALLE_EVALUACION_FACTOR (uno por factor).
- 61 registros en RESPUESTA_EVALUACION (uno por subfactor).

Pestaña 2 — Matriz de Evaluación (RF-04)

- Lista los 18 factores agrupados por dimensión TOE.
- Por cada factor:
- Muestra IS (Sugerida), ID (Decisor) e IR (Relativa) en tiempo real.
- Permite ajustar la evaluación del decisor (escala 1–4).
- Si IR = Irrelevante, no muestra subfactores (igual que GUIOS PRO).
- Si IR ≥ Opcional, muestra los subfactores con escala Likert 1–4.
- Autosave en cada cambio → POST /api/evaluaciones/autosave/.
- Indicador flotante: Autoguardando... / Guardado / Error.
- Buscador y filtros (Todos / Pendientes / Completados) sobre subfactores relevantes.
- Progreso calculado solo sobre subfactores de factores con IR ≥ Opcional.
- Botón "Procesar Inferencia & Ver Dictamen" → dispara el motor.

Pestaña 3 — Dictamen & Matriz FODA (RF-05 / RF-07)

- KPIs FODA (conteo de Fortalezas, Oportunidades, Debilidades, Amenazas).
- Promedios dimensionales TOE (T, O, E) en porcentaje.
- Gráfico radar de cobertura TOE.
- Tarjeta del dictamen oficial (CLASE A / B / C).
- Tabla detallada de factores con clasificación FODA.
- Exportación a PDF vía react-to-print (diálogo de impresión del navegador).

6. De dónde salen los datos: CSV, tesis y base de datos

Dos capas de datos

Capa	Origen	Cuándo se usa	Mutable por el usuario
Catálogo TOE	CSV + tesis (Cap. IV y V)	Siempre — estructura fija	No (solo admin)
Evaluación activa	Respuestas del evaluador	Por cada auditoría	Sí

Archivos CSV (semilla del catálogo)

Ubicación: backend/core/data/

Archivo	Contenido
factors.csv	18 factores con Importancia Sugerida (IS) y Alcance (Interno/Externo/Ambos)

guiosad_data.csv	61 subfactores (enunciados de preguntas) agrupados por dimensión y factor
------------------	---

Carga inicial a PostgreSQL

```
# Desarrollo local docker compose run --rm backend python manage.py seed_toe # Producción (Vercel): se ejecuta automáticamente en build.py si el catálogo está vacío
```

El comando `seed_toe.py` lee los CSV y crea:

- 3 DimensionTOE
- 18 Factor (con `importancia_sugerida` y `alcance`)
- 61 Subfactor

Los CSV ya no se leen en tiempo de ejecución. Solo sirven para poblar la base de datos. Toda la lógica posterior opera sobre PostgreSQL.

7. Los 5 indicadores de importancia (Capítulo V)

La tesis define 5 indicadores (Tabla 5.2, Sección 5.2.3):

Indicador	Símbolo	Escala	Origen en la tesis	En TechCheck Pro
Importancia del experto	IE	1–4	Encuesta a 57 expertos FLOSS	Precargado en <code>factors.csv</code>
Importancia de la literatura	IL	1–4	Ecuaciones 5.1 y 5.2	Precargado en <code>factors.csv</code>
Importancia sugerida	IS	1–4	Matriz IE × IL (Figura 5.10)	<code>Factor.importancia_sugerida</code>
Importancia del decisor	ID	1–4	Lo asigna el evaluador	<code>DetalleEvaluacionFactor.i</code>
Importancia relativa	IR	1–4	Matriz IS × ID (Figura 5.10)	Calculada por el motor y en vivo en la UI

Escala común

Valor	Etiqueta
1	Irrelevante
2	Opcional
3	Importante
4	Fundamental

Cálculo de IR — matriz Figura 5.10

```
# guiosad_calculos.py – réplica de update_results() en guiosad.html r = (is_n - 1 + id_n - 1) // 2 #  
índices 0-based IR = r + 1 # valor 1-4
```

Ejemplo: Compatibilidad con IS=3 (Importante) e ID=2 (Opcional):

$r = \text{floor}((2 + 1) / 2) = 1 \rightarrow \text{IR} = 2$ (Opcional). Coincide con la celda (3, 2) de la matriz.

8. Pantalla de evaluación: decisor, IR y subfactores

Al abrir un factor (ej. Compatibilidad) la UI muestra:

Sugerida: Importante Decisor: [Irrelevante | Opcional | Importante | Fundamental] Relativa: Opcional

- **Sugerida (IS):** viene del catálogo (factors.csv); el evaluador no la modifica.
- **Evaluación del Decisor (ID):** escala 1–4 de relevancia organizacional. Valor inicial = 1 (Irrelevante).
- **Importancia Relativa (IR):** recalculada en vivo al cambiar ID.
- **Ocultamiento:** si IR = Irrelevante, no se listan subfactores.

No confundir: Likert 1 en subfactores = "No cumple" (pendiente). ID 1 en el decisor = "Irrelevante para esta organización".

9. Evaluación de subfactores (escala Likert)

Valor	Significado
1	No cumple el requisito
2	Desconoce / No sabe
3	Cumple parcialmente
4	Cumple totalmente el requisito

Valor por defecto = 1 al iniciar una evaluación (placeholder de "aún no evaluado").

- Puntaje $\leq 1 \rightarrow$ **Pendiente**
- Puntaje $> 1 \rightarrow$ **Completado**

Si calculas el dictamen sin cambiar los 1, el motor interpreta todo como "No cumple" $\rightarrow$ PM bajo $\rightarrow$ CLASE C.

10. Cálculos del motor GUIOSAD paso a paso

Archivos: guiosad_calculos.py (fórmulas) + guiosad_engine.py (persistencia).

Paso	Descripción	Estado
1	Importancia Relativa (IR)	■ Matriz Figura 5.10

2	Selección de factores relevantes (IR ≥ 2)	■ Excluye Irrelevante
3	Ponderación de subfactores (PS) Likert 1–4	■ Correcto
4	Ponderación Media del factor (PM)	■ Correcto
5	Promedios globales TOE (T, O, E)	■ Solo factores relevantes
6	Dictamen A / B / C	■ resolver_dictamen()

11. Matriz FODA (Ecuación 5.3 de la tesis)

Tesis — Ecuación 5.3:

- **Fortaleza** si $PM \geq 3$ e impacto = interno
- **Oportunidad** si $PM \geq 3$ e impacto = externo
- **Debilidad** si $PM < 3$ e impacto = interno
- **Amenaza** si $PM < 3$ e impacto = externo

```
def clasificar_foda(promedio_likert, alcance): es_alto = promedio_likert >= 3.0 if alcance == 'Interno': return 'Fortaleza' if es_alto else 'Debilidad' return 'Oportunidad' if es_alto else 'Amenaza' # Externo o Ambos
```

Elemento	Estado
Umbral $PM \geq 3$	■ Correcto
Interno → F/D	■ Correcto
Externo → O/A	■ Correcto
Factor Soporte (Ambos)	■ Pendiente: selector Interno/Externo en UI

12. Dictamen final: Clase A, B y C

Clase	Significado	Color en UI
CLASE A	Adoptar el FLOSS	Verde
CLASE B	Adoptar condicionado (con matices)	Ámbar
CLASE C	Rechazar / Posponer la adopción	Rojo

Regla implementada (resolver_dictamen):

Condición	Dictamen
-----------	----------

Debilidad/Amenaza con IR = Importante o Fundamental	CLASE C
Debilidad/Amenaza con IR = Opcional (y ninguna crítica)	CLASE B
Ninguna Debilidad/Amenaza en factores evaluados	CLASE A

13. Paridad backend ↔ frontend ↔ GUIOS PRO

Cálculo / Comportamiento	Python	JavaScript	Estado
Normalizar escala 1–4	<code>normalizar_nivel()</code>	<code>normalizarNivel()</code>	■
IR	<code>calcular_importancia_relativa()</code>	<code>calcularImportanciaRelativa()</code>	■
Excluir Irrelevante	<code>es_factor_relevante()</code>	<code>esFactorRelevante()</code>	■
Etiquetas IS/ID/IR	<code>etiqueta_importancia()</code>	<code>etiquetaImportancia()</code>	■
FODA	<code>clasificar_foda()</code>	— (solo backend)	■
Dictamen	<code>resolver_dictamen()</code>	— (solo backend)	■
ID inicial = 1	<code>views.py</code> al iniciar	<code>Evaluation.jsx</code> fallback ?? 1	■
Ocultar subfactores si IR=1	Motor excluye	<code>FactorDetailPanel</code> oculta UI	■

14. Verificación Cap. V vs implementación actual

Cálculo / Regla	Estado	Notas
IE — encuesta expertos	■ Precargado	Resulta en factor
IL — ecuaciones 5.1, 5.2	■ Precargado	Resulta en factor
IS — matriz IE×IL	■ Precargado	Campo import
ID — decisor	■ Implementado	Wizard + autosav escala 1–4

IR — matriz IS×ID	■ Implementado	Fórmula floor (
Filtro IR > Irrelevante	■ Implementado	Backen + UI ocultan subfact
PS — Likert 1–4	■ Correcto	
PM — promedio subfactores	■ Correcto	
FODA — ecuación 5.3	■ Correcto	Umbral PM≥3
Dictamen A/B/C	■ Implementado	14 tests OK
Soporte Interno/Externo	■ Pendiente	Alcance siempre como Externo
Promedios TOE (T, O, E)	■ Extensión	Dashbo
Exportación PDF	■ Implementado	react- + endpoi servido

Resumen: Núcleo IR + FODA + dictamen alineado con guiosad.html / GUIOS PRO.

15. API REST disponible

Todas las rutas de evaluación requieren JWT y filtran por usuario autenticado.

Autenticación

Método	Endpoint	Descripción	RF
POST	/api/token/	Login — obtiene JWT	RF-01
POST	/api/token/refresh/	Renovar token	RF-01
GET	/api/perfil/	Perfil del usuario autenticado	RF-01

Catálogo y evaluaciones

Método	Endpoint	Descripción	RF
GET	/api/catalogo/	Catálogo TOE (18 factores, 61 subfactores)	RF-02
POST	/api/evaluaciones/iniciar/	Crear/recuperar evaluación	RF-03
GET	/api/evaluaciones/misfactores/	Historial del usuario	RF-03
GET	/api/evaluaciones/<id>/	Cargar progreso guardado	RF-04
POST	/api/evaluaciones/autosave/	Guardar Likert + importancias (1–4)	RF-04
POST	/api/evaluaciones/calcular/	Ejecutar motor FODA + dictamen	RF-05
POST	/api/evaluaciones/bloquear/	Bloquear evaluación	RF-06
POST	/api/evaluaciones/reabrir/	Reabrir evaluación	RF-06
POST	/api/evaluaciones/archivar/	Archivar evaluación	RF-06
POST	/api/evaluaciones/comparar/	Comparar dos evaluaciones	RF-08
POST	/api/evaluaciones/simular/	Simular escenario what-if	RF-09
GET	/api/evaluaciones/<id>/pdf	Exportar PDF del dictamen	RF-07
GET	/api/evaluaciones/<id>/auditoria/	Historial de auditoría	RF-10
GET	/api/evaluaciones/alertas/	Alertas de progreso	RF-11
GET	/api/dashboard/ejecutivo/	Dashboard ejecutivo	RF-12

Administración

Método	Endpoint	Descripción
GET/POST	/api/admin/usuarios/	Listar/crear usuarios
PATCH	/api/admin/usuarios/<id>/	Actualizar usuario
GET/POST	/api/admin/dimensiones/	Gestión de dimensiones TOE
GET/POST/PATCH	/api/admin/factores/	Gestión de factores
GET/POST/PATCH/DELETE	/api/admin/subfactores/	Gestión de subfactores

16. Modelo de datos (PostgreSQL)

USUARIO ■■■■■■■■■■■■■■■■■■■■ ■ SOFTWARE_OBJETIVO ■■■■■■■■■■ EVALUACION ■■■■■■■■ RESPUESTA_EVALUACION (Likert por subfactor) ■ ■ ■ ■■■■ DETALLE_EVALUACION_FACTOR (ID, IR, FODA) ■ DIMENSION_TOE ■■■■ FACTOR ■■■■ SUBFACTOR (IS, alcance)

Tabla	Qué guarda
Factor	Catálogo: nombre, IS, alcance, dimensión
Subfactor	Catálogo: enunciado de pregunta
Evaluacion	Cabecera: usuario, software, estado, promedios T/O/E, dictamen
RespuestaEvaluacion	Likert 1–4 por subfactor en cada evaluación
DetalleEvaluacionFactor	ID del decisor, IR calculada, resultado FODA (null si irrelevante)

17. Comandos útiles para desarrollo

Entorno local (Docker)

```
# Levantar entorno completo docker compose up # Poblar catálogo TOE desde CSV docker compose run --rm backend python manage.py seed_toe # Crear usuario evaluador docker compose run --rm backend python manage.py create_evaluador # Crear usuario administrador docker compose run --rm backend python manage.py create_admin # Ejecutar tests del motor y API (14 tests) docker compose run --rm backend python manage.py test core.tests # Instalar dependencias frontend cd frontend && npm install # Build de producción frontend cd frontend && npm run build
```

Despliegue en Vercel

```
# Despliegue automático (lee variables de backend/.env) ./scripts/deploy-vercel.sh # Despliegue manual npx vercel --prod --yes # Regenerar documentación PDF ./scripts/generar-documentacion-pdf.sh
```

URLs locales

Servicio	URL
Frontend	http://localhost:3000
Backend API	http://localhost:8000/api
Admin Django	http://localhost:8000/admin

18. Solución de problemas frecuentes

Pantalla negra: "does not provide an export named 'default'"

Error de caché corrupta de Vite tras un error de sintaxis previo.

```
docker exec techcheck_react rm -rf node_modules/.vite docker compose restart frontend # Recarga dura en el navegador: Ctrl+Shift+R
```

Error "Failed to resolve import react-to-print"

```
docker compose restart frontend
```

Dictamen inesperado con todo en "No cumple"

Si no se cambiaron los Likert por defecto (valor 1), el motor interpreta *No cumple* en todos los subfactores → PM bajo → CLASE C. Califica al menos los subfactores de factores relevantes ($IR \geq$ Opcional) antes de calcular.

No hay evaluación activa

Selecciona o crea un proyecto en la pestaña Panel de Control. Sin `activeEvalId` en `localStorage`, las pestañas 2 y 3 muestran error.

Error de base de datos en Vercel

Verifica que `DATABASE_URL` esté configurada en el dashboard de Vercel (Storage → Neon). La base de datos local de Docker no es accesible desde los servidores de Vercel.

Despliegue falla tras push a GitHub

Asegúrate de que los archivos de configuración de Vercel estén en el repositorio:

- `vercel.json` (raíz)
- `backend/pyproject.toml`
- `backend/build.py`

19. Despliegue en producción (Vercel + Neon)

Arquitectura de despliegue

El proyecto usa **Vercel Services** para desplegar frontend y backend en un solo dominio:

Servicio	Directorio	Tecnología	Ruta pública
frontend	frontend/	React + Vite	/
backend	backend/	Django (serverless)	/api/

Configuración en `vercel.json` (raíz del proyecto).

Requisitos previos

1. Cuenta en [Vercel](#) vinculada a GitHub.
2. Base de datos PostgreSQL en la nube (recomendado: **Neon** vía integración de Vercel).
3. Repositorio conectado: `SnayderC/TechCheckPro`.

Variables de entorno (producción)

Variable	Descripción	Ejemplo
SECRET_KEY	Clave secreta de Django	Generada automáticamente
DEBUG	Modo depuración	False
DATABASE_URL	Conexión PostgreSQL (Neon)	Auto-configurada por integración
ADMIN_USERNAME	Usuario admin inicial	admin
ADMIN_PASSWORD	Contraseña admin inicial	Admin2026!
VITE_API_URL	URL base de la API para el frontend	/api
ALLOWED_HOSTS	Hosts permitidos	localhost, .vercel.app

Las variables locales se definen en backend/.env. En Vercel se configuran en **Project Settings** → **Environment Variables** o mediante ./scripts/deploy-vercel.sh.

Proceso de build en Vercel

1. **Frontend:** npm run build → archivos estáticos en frontend/dist/.

Backend:

- Instala dependencias desde backend/pyproject.toml.
- Ejecuta python build.py:
 - Migraciones de base de datos (migrate --noinput).
 - Seed del catálogo TOE si está vacío (seed_toe).
 - Creación del usuario admin si ADMIN_USERNAME y ADMIN_PASSWORD están definidos.
 - Despliega Django como función serverless (WSGI).

Despliegue automático desde GitHub

Al hacer **push** a la rama main, Vercel despliega automáticamente a producción:

```
git add . git commit -m "descripción del cambio" git push origin main
```

- Push a main → producción (techcheck-pro.vercel.app)
- Push a otra rama → preview (URL temporal)

Despliegue manual con CLI

```
# 1. Iniciar sesión npx vercel login # 2. Vincular proyecto (solo la primera vez) npx vercel link --yes --project techcheck-pro # 3. Provisionar base de datos Neon (solo la primera vez) npx vercel integration add neon --plan free_v3 -n techcheck-db # 4. Desplegar npx vercel --prod --yes
```

Script de despliegue automatizado

```
./scripts/deploy-vercel.sh
```

Este script:

1. Verifica sesión en Vercel.
2. Vincula el proyecto si no está vinculado.
3. Sube variables de entorno desde backend/.env.

4. Despliega a producción.

Credenciales de producción actuales

Campo	Valor
URL	https://techcheck-pro.vercel.app
Usuario	admin
Contraseña	Admin2026!

Archivos clave del despliegue

Archivo	Función
vercel.json	Configuración de servicios y rewrites
backend/pyproject.toml	Dependencias Python y entrypoint WSGI
backend/build.py	Script de build (migraciones, seed, admin)
backend/techcheck/settings.py	Configuración Django (DB, CORS, JWT)
scripts/deploy-vercel.sh	Script de despliegue automatizado
.vercelignore	Archivos excluidos del despliegue

Diagrama resumen del flujo de cálculo

[illegible]

Referencias

- **Tesis:** *Factores para la adopción de soluciones basadas en software libre* — Capítulos V (indicadores y FODA) y VI (GUIOS / GUIOS PRO).
- **Prototipo:** `guiosad.html` — referencia de cálculo y flujo GUIOS PRO.
- **Documento académico S7:** Componente Práctico — Requerimientos RF-01 a RF-15.
- **Rincón, Mazo & Salinesi [116]** — APPLIES framework (matriz de importancia, Fig. 5.10).

Documento actualizado para el proyecto GUIOSPRO_FLOSS / TechCheck Pro. Alineado con `guiosad.html`, `backend/core/utlils/guiosad_calculos.py`, `backend/core/utlils/guiosad_engine.py`, `frontend/src/utlils/guiosadCalculos.js`, `frontend/src/pages/Evaluation.jsx` y `frontend/src/pages/Report.jsx`.